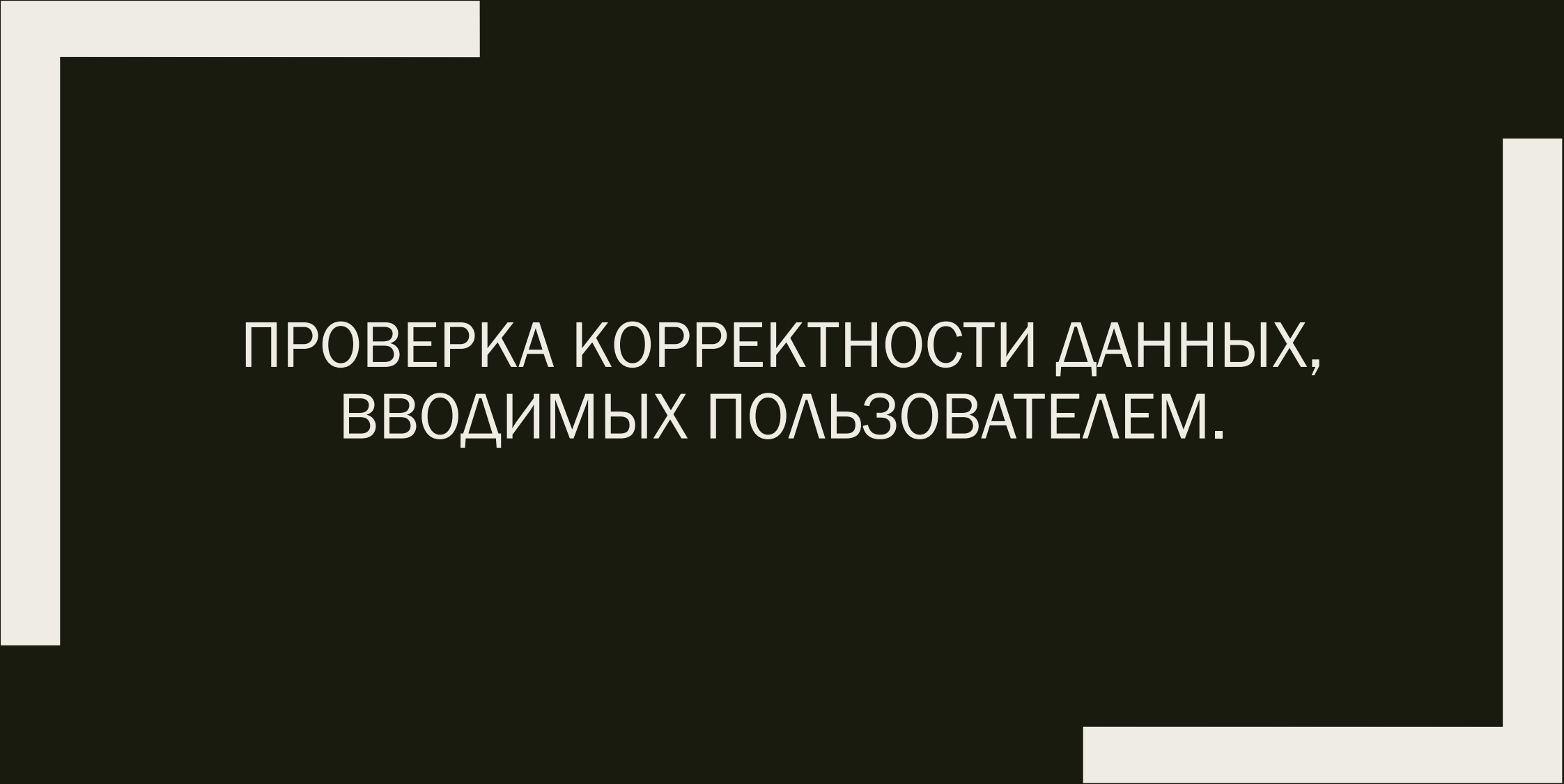


A decorative frame composed of thick white lines. It starts with a horizontal line at the top left, followed by a vertical line extending downwards. At the bottom right, there is another horizontal line extending to the right, followed by a vertical line extending upwards. These lines form an open rectangular shape that frames the central text.

ПРОВЕРКА КОРРЕКТНОСТИ ДАННЫХ,
ВВОДИМЫХ ПОЛЬЗОВАТЕЛЕМ.

Проверка корректности данных, ВВОДИМЫХ ПОЛЬЗОВАТЕЛЕМ.

Проверке корректности данных, вводимых пользователем необходимо уделять достаточно большое внимание, поскольку необработанные ошибки, возникающие при неправильном вводе данных, приводят к ошибкам в работе скрипта. Например, мы создаём форму для отправки пользователем письма, при этом адрес электронной почты необходимо вводить пользователю. В этом случае, для корректной работы программы мы должны сделать две вещи:

- ❑ Проверить, что поле, в которое заносится электронный адрес непустое (поскольку пользователь может просто забыть ввести адрес, и, если этот случай необработан, возникнет ошибочная ситуация);
- ❑ Проверить соответствие введенного адреса с помощью регулярного выражения.

Проверка на пустоту поля

В следующем примере проверяется три строки и определяется, имеет ли каждая строка значение, является ли пустой строкой или имеет значение null:

ДЛЯ C#:

```
string targetString = null;  
var result = String.IsNullOrEmpty(targetString);
```

ДЛЯ PHP:

```
empty($var);
```

Проверка допустимости вводимых данных

Представим, что нам надо проверить данные формы обратной связи. Как правило, такая проверка осуществляется при помощи регулярных выражений. Рассмотрим пример, в котором создается регулярное выражение для проверки адреса электронной почты.

Адрес должен иметь вид Walls.234@mail.ru. У адреса две составляющие – имя пользователя и имя домена, которые разделены знаком “@”. В имени пользователя могут присутствовать буквы нижнего и верхнего регистров, цифры, знаки подчеркивания, минуса и точки.

Для проверки разделителя между именем пользователя и именем домена в выражение требуется добавить +@. Регулярное выражение, проверяющее имя пользователя и наличие разделителя имеет вид:

```
"/[0-9a-z_\.]+@[0-9a-z_^\.]"
```

Проверка файлов

Для файловых полей используются те же самые проверки, что и для обычных, однако они применяются уже не к собственно содержимому поля, а к массиву `$_FILES`, поэтому инструкции для файловых полей должны иметь специальную структуру. Рассмотрим в качестве примера следующую HTML-форму:

```
<form method="POST" enctype="multipart/form-data">  
  ...  
  <input type="file" name="photo">  
  ...  
</form>
```

Проверка файлов

Массив `$_FILES` для такой формы в случае успешной загрузки файла будет иметь такой вид:

```
Array
{
    [photo] => Array
        {
            [name] => britney 100x100.jpg
            [type] => image/jpeg
            [tmp_name] => /tmp/php1AD.tmp
            [error] => 0
            [size] => 2752
        }
}
```

Что такое валидация формы?

Откройте любой популярный сайт с формой регистрации и вы заметите, что они дают вам обратную связь, когда вы вводите ваши данные не в том формате, который они ожидают от вас. Вы получите подобные сообщения:

- ❑ "Это поле обязательно для заполнения" (вы не можете оставить это поле пустым)
- ❑ "Пожалуйста введите ваш телефонный номер в формате xxx-xxxx" (вводит три цифры разделенные тире, за ними следуют четыре цифры)
- ❑ "Пожалуйста введите настоящий адрес электронной почты" (если ваша запись не в формате "somebody@example.com")
- ❑ "Ваш пароль должен быть от 8 до 30 символов длиной, и содержать одну заглавную букву, один символ, и число"

Что такое валидация формы?

Это называется **валидация формы** — когда вы вводите данные, **веб-приложение** проверяет, что данные корректны. Если данные верны, **приложение** позволяет **данным быть отправленными на сервер** и (как правило) быть сохраненными в базе данных; **если нет** - оно **выдает вам сообщение об ошибке**, объясняющее какие исправления необходимо внести.

Что такое валидация формы?

Проверка формы может быть реализована несколькими различными способами.

Мы хотим сделать заполнение веб-форм максимально простым. Итак, почему мы настаиваем на подтверждении наших форм? Существуют три основные причины:

- ❑ **Мы хотим получить нужные данные в нужном формате** — наши приложения не будут работать должным образом, если данные нашего пользователя хранятся в неправильном формате, если они вводят неправильную информацию или вообще не передают информацию.
- ❑ **Мы хотим защитить учетные записи наших пользователей** — заставляя наших пользователей вводить защищенные пароли, это упрощает защиту информации об их учетной записи.
- ❑ **Мы хотим обезопасить себя** — существует множество способов, которыми злоумышленники могут злоупотреблять незащищенными формами, чтобы повредить приложение, в которое они входят.

Различные типы валидации формы

Существует два разных типа проверки формы, с которыми вы столкнетесь в Интернете:

- ❑ Проверка на стороне клиента - это проверка, которая происходит в браузере, прежде чем данные будут отправлены на сервер. Это удобнее, чем проверка на стороне сервера, так как дает мгновенный ответ. Ее можно далее подразделить на:
 - ❑ *JavaScript проверка выполняется с использованием JavaScript. Полностью настраиваемая.*
 - ❑ *Встроенная проверка формы, используя функции проверки формы HTML5. Для этого обычно не требуется JavaScript. Встроенная проверка формы имеет лучшую производительность, но она не такая настраиваемая, как с использованием JavaScript.*

Различные типы валидации формы

- ❑ Проверка на стороне сервера - это проверка, которая возникает на сервере после отправки данных. Серверный код используется для проверки данных перед их сохранением в базе данных. Если данные не проходят проверку валидности, ответ отправляется обратно клиенту, чтобы сообщить пользователю, какие исправления должны быть сделаны. Проверка на стороне сервера не такая удобная, как проверка на стороне клиента, поскольку она не выдает ошибок до тех пор, пока не будет отправлена вся форма. Тем не менее, проверка на стороне сервера - это последняя линия защиты вашего приложения от неправильных или даже вредоносных данных. Все популярные серверные фреймворки имеют функции для проверки и очистки данных (что делает их безопасными).

Использование встроенной проверки формы

Одной из особенностей HTML5 является возможность проверки большинства пользовательских данных без использования скриптов. Это делается с помощью атрибутов проверки элементов формы, которые позволяют вам указывать правила ввода формы, например, нужно ли заполнять значение, минимальная и максимальная длина данных, должно ли это быть число, адрес электронной почты, адрес или что-то еще, и шаблон, которому это должно соответствовать. Если введенные данные соответствуют всем этим правилам, данные считаются валидными; если нет - невалидными.

Использование встроенной проверки формы

Когда элемент валидный:

- ❑ Элемент соответствует CSS псевдо-классу `:valid` ; это позволяет вам применить конкретный стиль к валидным элементам.
- ❑ Если пользователь пытается отправить данные, браузер отправит форму, если нет ничего, остановит отправку.

Если элемент невалидный:

- ❑ Элемент соответствует CSS псевдо-классу `:invalid`; это позволяет вам применить конкретный стиль к невалидным элементам.
- ❑ Если пользователь пытается отправить данные, браузер заблокирует форму и выдаст сообщение об ошибке.

Ограничения проверки элементов `input` - простое начало

В этом разделе мы рассмотрим некоторые функции HTML5, которые можно использовать для проверки `<input>` элементов.

Начнем с простого примера - `input`. Он включает простой текст `<input>` соответствующий ярлык (`label`) и отправку (`submit`) `<button>`.

Требуемый атрибут (required)

Простейшей функцией проверки HTML5 для использования является **required** атрибут. Если вы хотите сделать ввод обязательным, вы можете пометить элемент, используя этот атрибут. **Если этот атрибут установлен, форма не будет отправляться (и будет отображаться сообщение об ошибке), когда поле пустое (поле input также будет считаться невалидным).**

Проверка с регулярными выражениями

Другой очень распространенной функцией проверки является **атрибут `pattern`**, который ожидает **Regular Expression** в качестве значения. Регулярное выражение (**regex**) - это шаблон который используется для соответствия комбинаций символов в текстовых строках, поэтому он идеально подходит для проверки формы (а также для многих других целей в JavaScript).

Проверка с регулярными выражениями

```
<form>
  <label for="choose">Would you prefer a banana or a cherry?</label>
  <input id="choose" name="i_like" required pattern="banana|cherry">
  <button>Submit</button>
</form>
```

В этом примере `<input>` элемент принимает одно из двух возможных значений: строку "banana" или строку "cherry".

Ограничение длины ваших записей

Все текстовые поля, созданные с помощью элементов (`<input>` или `<textarea>`) могут быть ограничены по размеру, используя атрибуты `minlength` и `maxlength`. Поле не валидно если его значение короче чем `minlength` или значение длиннее значения `maxlength`. Браузеры часто не позволяют пользователю вводить более длинное значение, чем ожидалось, в текстовые поля в любом случае, но полезно иметь этот мелкозернистый элемент управления.

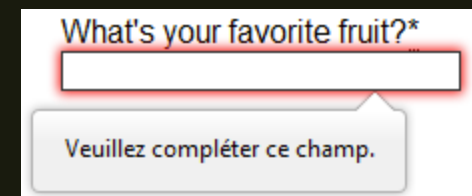
Для числовых полей (например `<input type="number">`), атрибуты `min` и `max` также обеспечивают ограничение валидации. Если значение поля меньше атрибута `min` или больше атрибута `max`, поле будет невалидным.

Индивидуальные сообщения об ошибках

Как видно из приведенных выше примеров, каждый раз, когда пользователь пытается отправить невалидную форму, браузер отображает сообщение об ошибке. Способ отображения этого сообщения зависит от браузера.

Эти автоматизированные сообщения имеют два недостатка:

- ❑ Нет стандартного способа изменить внешний вид используя CSS.
- ❑ Они зависят от языка браузера, что означает, что у вас может быть страница на одном языке, но сообщение об ошибке отображаться на другом языке.



Индивидуальные сообщения об ошибках

Чтобы настроить внешний вид и текст этих сообщений, вы должны использовать JavaScript; нет способа сделать это, используя только HTML и CSS.

В JavaScript вы вызываете метод `setCustomValidity()`:

```
var email = document.getElementById("mail");

email.addEventListener("input", function (event) {
  if (email.validity.typeMismatch) {
    email.setCustomValidity("I expect an e-mail, darling!");
  } else {
    email.setCustomValidity("");
  }
});
```